



中华人民共和国通信行业标准

XX/T s—XXXX

研发智能化能力成熟度模型

Developing an intelligent capability maturity model

[点击此处添加与国际标准一致性程度的标识]

(征求意见稿)

2025 年 12 月 2 日

XXXX - - XX 发布

XXXX - XX - 实施

中华人民共和国工业和信息化部 发布

目次

前 言II

研发智能化能力成熟度模型1

1. 范围1

2. 规范性引用文件1

3. 术语和定义1

 3.11

 3.21

4. 缩略语1

5. 研发智能化能力成熟度模型架构.....2

6. 研发智能化能力成熟度模型分级规则.....2

 6.1. 分级概述.....2

 6.2. 分级规则.....3

7. 智能能力3

 7.1. 工程理解（新增）3

 7.2. 智能编码（新增）4

 7.3. 开发者辅助（新增）11

 7.4. 智能评审.....14

 7.5. 辅助 CI/CD.....15

8. 能力应用16

 8.1. 企业研发管理.....16

 8.2. 智能能力应用.....17

 8.3. 研发度量智能化.....19

前 言

本文件按照GB/T 1.1—2020《》给出的规则起草。

请注意本文件的某些内容可能涉及专利。本文件的发布机构不承担识别这些专利的责任。

本文件由中国通信标准化协会提出并归口。

本文件起草单位：xxx

本文件主要起草人：xxx、xxx等人

研发智能化能力成熟度模型

1. 范围

本标准规范了研发智能化能力的完整框架，覆盖从战略到实践的全面维度。主要包括：智能能力、能力应用、技术平台 3 大核心能力。聚焦于研发活动中人机协同的智能化水平，旨在为组织提供系统化的能力发展指引，既关注技术工具的引入与应用，也强调与之适配的组织、流程、管理与度量体系的协同演进。

本标准适用于通过智能化技术提升研发效能、质量和创新能力的软件与信息技术服务相关的组织。

应用方：用于评估和提升其智能化应用水平；

建设方：用于评估和指导其智能化工具、平台或服务的规划、构建与运营能力；

该标准可为各类组织制定智能化转型战略、开展能力差距分析、规划发展路径以及实施评估改进提供通用框架和基准参考。

2. 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件，仅所注日期的版本适用于本文件。凡是不注日期的引用文件，其最新版本（包括所有的修改单）适用于本文件。

3. 术语和定义

下列术语及定义适用于本文件

3.1

智算云 Intelligent computing cloud
以 AI 算力为核心引擎，聚焦算力资源全面高效调度资源与运维能力。

3.2

算力运营 computing power operation
对智算资源的调度、监控及效率提升进行管理和优化的运营活动。

4. 缩略语

下列缩略语适用于本文件：

GPU：图形处理器（Graphics Processing Unit）
NPU：神经网络处理单元（Neural Processing Unit）

5. 研发智能化能力成熟度模型架构

本文件所定义的研发智能化能力成熟度模型架构，如图1所示：



图 1 研发智能化能力成熟度模型架构

6. 研发智能化能力成熟度模型分级规则

6.1. 分级概述

研发智能化能力成熟度模型，共分为3个级别，每个级别中按照不同程度说明。各级别对应的具体能力要求见第7-9章。各级别技术要求描述如下：

级别	能力概述
基础级（1级）	实现研发过程 工具化、点状辅助 。组织引入独立的智能编码工具（如代码补全、基础检查），在部分场景替代重复劳动。技术平台主要依赖外部公有模型或基础开源方案，以API调用为主。价值体现在提升单点效率。
增强级（2级）	实现研发过程 流程化、体系协同 。组织将多种智能化能力（如生成、检查、问答）系统化集成至标准研发流程中，实现关键场景的流程嵌入与人机协同。建立了治理机制与效能度量体系，能够追踪智能化应用对研发效率与质量的提升。技术平台开始构建私有化能力，进行场景数据微调和工程化集成，支持规模化应用。
先进级（3级）	实现研发过程 生态化、智能驱动 。智能化与业务目标深度对齐，形成全链路、自适应优化能力，并催生新的研发模式。建立了基于数据的价值度量与反馈闭环，

	驱动技术平台与业务实践持续协同进化。技术平台具备领先的自主演进能力，通过高质量数据和领域模型驱动，实现智能化能力的预测、决策与创新突破。
--	--

6.2. 分级规则

测试智能体能力成熟度模型，能力要求项分为1级、2级、3级。

满足90%及以上的1级能力项，即可认为达到基础级要求；

满足90%及以上的1级能力项且满足90%及以上的2级能力项，即可认为达到增强级要求；

满足90%及以上的1级能力项、90%及以上的2级能力项且满足90%及以上的3级能力项，即可认为达到先进级要求。

可选项供参考，不作为必须满足项。

7. 智能能力

7.1. 工程理解（新增）

7.1.1. 工程索引

- 1 级：
- 1) 应支持调用 `grep` 等命令行工具在工作区搜索代码；
 - 2) 宜建立全仓库的结构化索引，建立依赖关系图谱，理解代码间的深层依赖和调用关系。
- 2 级：
- 1) 应支持获取指定代码文件的文件引用、可引用的函数签名列表；
 - 2) 应支持清理指定文件中无效的文件引用；
 - 3) 应支持为指定文件补全缺失的文件引用。
- 3 级：
- 1) 应支持通过向量化等技术建立对全仓库核心文件的索引；
 - 2) 应支持基于关键词、语义的搜索代码和文档召回；
 - 3) 应支持提供代码片段实现相似代码搜索和文档召回。

7.1.2. 架构理解

- 1 级：
- 1) 应能提供仓库的目录文件树展示，以及大文件树的压缩展示；

- 2) 应能提供工程的描述信息，对于开发类工程，提供主要及辅助的编程语言、实现的功能等描述信息。

2 级：

- 1) 应能分析并提供当前项目的模块划分、入口文件、开发框架（如前后端分离、MVC、微服务架构等）；
- 2) 应能分析并提供当前项目的构建工具、构建配置、构建命令和 CI/CD 流水线操作信息；
- 3) 应能识别项目的版本演进方案和版本变更的管理信息。

3 级：

- 1) 应能针对指定需求生成需求分析文档、设计文档、开发步骤分解文档等，以支持规约驱动开发模式。
- 2) 应支持跨项目的工程理解。

7.1.3. 质量规范理解

1 级：

——应能识别并提供当前项目的编码风格和代码规范。

2 级：

——应能识别并提供当前项目的单元测试架构和测试命令。

3 级：

——应能识别并提供当前项目的安全相关编码规范和约束。

7.2. 智能编码（新增）

7.2.1. 代码生成

1 级：

- 1) 应支持基于自然语言描述生成单函数/单模块代码，支持 Java、Python、JavaScript 等常用语言；
- 2) 应生成代码时、自动带出注释（函数说明等）；
- 3) 应具备生成代码的可编译能力；
- 4) 应具备局部上下文感知能力，能识别当前文件中的变量、函数和类定义。

2 级：

- 1) 应支持函数/类、单个完整文件（如 Controller 类、Vue 组件、工具类）、简单模块的代码生成，支持跨函数的逻辑关联；
- 2) 应支持基于项目级上下文（如已有模块、接口契约、依赖库）生成语义一致；
- 3) 应支持多轮交互式生成模式，允许用户通过自然语言反馈修正或细化生成结果；
- 4) 应支持生成符合行业通用规范的代码格式；
- 5) 宜提供生成代码的可解释性说明，包括核心逻辑依据、关键变量作用及潜在边界条件。

3 级：

- 1) 应支持多文件/跨模块、工程级别、跨语言协同代码生成；
- 2) 应支持自定义编码规范、安全规范等，代码智能生成能够遵守自定义代码规范；
- 3) 应支持根据图片、多文件、项目管理平台上需求点智能生成需求功能代码，可运行；
- 4) 宜具备演化学习机制，能根据用户采纳/拒绝行为/代码评审等结果持续优化生成策略；
- 5) 宜支持特定领域（如金融、军队、政府等）合规性代码生成。

7.2.2. 代码续写

1 级：

- 1) 应支持常见编程语言（如 Python、Java、JavaScript、C++、Go、TypeScript 等）代码的续写；
- 2) 应支持单行/短句语法补全；
- 3) 应支持基于关键字触发的续写功能；
- 4) 应支持输出通用代码模板（如无参数校验的空函数体、for/while 循环的语法框架、数据库查询的基础 SQL 嵌入代码等）。

2 级：

- 1) 应支持基于语义的多行代码续写。能够结合代码上下文与文档注释，输出符合业务逻辑的函数/方法体；
- 2) 应支持续写完整代码块，并可直接嵌入现有代码，实现成功编译运行；
- 3) 应支持基础编码规范适配，自动调整代码格式；
- 4) 应能识别简单逻辑缺口，续写时自动补充参数，实现非空校验、数组越界判断等。
- 5) 宜支持续写代码自动补充合规注释内容，如函数功能描述、参数含义与类型、返回值说明等。

3 级：

- 1) 应具备深度感知全局上下文能力，续写时自动复用现有工具类、适配数据模型；

- 2) 应支持复杂业务逻辑续写，能理解业务语义并生成符合规则的代码；
- 3) 应支持跨模块、跨服务的代码续写，能够结合上下游接口定义、数据模型和业务逻辑，完成跨组件的代码编写（如微服务间的调用逻辑、数据库与业务层的交互代码）；
- 4) 应具备续写-优化闭环能力（如续写代码后自动检测性能问题、语法冗余，生成优化建议；若存在逻辑冲突，自动调整续写方向）；
- 5) 应支持自定义编码规范、安全规范、数据库语法规则等，并根据规范要求自动续写。

7.2.3. 代码优化

1 级：

- 1) 应支持代码基础格式检测，符合主流语言编码规范；
- 2) 应检测并建议消除明显的代码坏味道（如过长函数、过大类、重复代码片段等）；
- 3) 应支持性能优化，识别并提示常见低效写法，可对高复杂度函数给出简化建议；
- 4) 应支持自动格式化代码，适配基础编码规范；
- 5) 应具有智能识别语法错误并提供修复建议功能。

2 级：

- 1) 应具备在 sql 查询、缓存策略、高并发等多维度性能优化能力；
- 2) 应具备适配框架特性优化（如 SpringBoot 配置优化、MyBatisSQL 语句优化）能力；
- 3) 应支持安全合规优化，自动修复或给出规避常见漏洞（如 SQL 注入风险、XSS 漏洞防护）建议；
- 4) 应支持自定义编码规范适配，可按需导入自定义规范并按规则批量自动优化代码。

3 级：

- 1) 应具备智能性能预测能力，基于代码逻辑与数据量预判性能瓶颈，提前给出优化方案；
- 2) 应支持在不改变业务结果的前提下，业务逻辑层面优化（如复杂规则引擎代码简化、状态机逻辑优化）；
- 3) 应具备结合项目历史优化数据，生成个性化优化策略；
- 4) 应支持生成优化前后的对比报告，可包含代码复杂度、执行效率、性能提升等维度评估；
- 5) 宜利用 AI 模型预测代码变更对资源消耗（CPU、内存、网络）的影响，并提供量化建议。

7.2.4. 代码解析

1 级：

- 1) 应支持解析主流编程语言（如 Java、Python、JavaScript、Go 等）的基础语法结构；
- 2) 应能识别代码中的类、方法/函数、变量、常量等基础元素；
- 3) 应支持识别代码中的控制流（如条件语句 if-else、分支语句 switch/case 等）与基础调用关系；
- 4) 应支持代码文件的语法树可视化展示；
- 5) 应能识别代码中的注释类型（如单行注释、块注释）并提取注释文本。识别基础的代码块结构和作用域范围。

2 级：

- 1) 应具备分析模块间、函数间的依赖关系，生成调用链路关系图谱；
- 2) 应能识别设计模式和架构模式的应用；
- 3) 应支持结合项目结构和配置文件理解代码上下文；
- 4) 应能关联数据库表结构与 ORM 映射代码，解析数据模型与表字段的对应关系；
- 5) 应支持代码变更差异解析，识别新增/删除/修改的代码元素及逻辑影响。

3 级：

- 1) 应支持多语言、跨文件的逻辑流程深度解析，识别业务规则对应的代码逻辑；
- 2) 应能解析分布式系统中的跨服务调用关系，生成全链路逻辑图谱；
- 3) 应支持非结构化代码元素解析，识别其语义与依赖；
- 4) 应能识别不同代码/实时增量信息的一致性校验；

7.2.5. 代码注释

1 级：

- 1) 应支持 java、js、python、go、c/c++等主流语言的函数基础注释生成；
- 2) 应支持基于函数名、变量名自动推测注释内容；
- 3) 应支持识别变量类型如参数、变量，生成变量注释；
- 4) 应具备注释语法错误校验、注释模板补全能力，支持注释格式标准化，自动调整注释排版，并支持识别未闭合注释、注释嵌套等语法问题；
- 5) 应具备手动触发注释生成能力。

2 级：

- 1) 应支持关联代码逻辑生成流程化注释（如循环逻辑、条件判断的语义说明）；
- 2) 应支持自定义注释模板，注释按照模版格式生成；
- 3) 应支持注释与代码语义的一致性校验；

- 4) 应支持注释自动更新，代码修改时同步调整关联注释内容；
- 5) 应具备自动识别已有注释的不规范问题的能力，并支持提供优化建议，可手动更新到工程文件代码中。

3 级：

- 1) 应支持多语言、跨模块/跨服务的关联注释生成；
- 2) 应具备结合业务场景生成语义化注释能力；
- 3) 应支持动态注释生成，根据代码运行场景、数据流向自动补充注释；
- 4) 应支持关联需求文档、接口文档生成注释；
- 5) 应支持注释翻译与多语言适配，可自动根据系统情况注释中英文版本；
- 6) 宜具备基于注释自动生成接口文档的能力，支持实现注释与文档实时同步。

7.2.6. 版本管理

1 级：

- 1) 应支持 Git/SVN 等版本管理工具的基础提交信息生成，基于代码变更内容自动推荐提交描述；
- 2) 应具有代码提交记录查询、版本差异比对等基础版本管理功能；
- 3) 应支持自动检测未提交代码、未推送分支，以及提醒功能；
- 4) 应能识别同一文件的合并冲突；
- 5) 应支持基础版本回滚指引，提供简单的回滚命令建议。

2 级：

- 1) 应具备研发过程的可追溯能力，支持关联版本与任务（如需求、缺陷）可追溯；
- 2) 应能分析跨文件或逻辑冲突，提供解决建议并支持部分自动化合并；
- 3) 应具备辅助分支管理，包括分支创建、合并、识别过期分支提示删除等；
- 4) 应支持生成分支演化图谱，可视化展示分支创建、合并、提交历史；
- 5) 应支持版本差异智能对比，展示核心变更点；
- 6) 应能关联项目管理工具，支持自动提取任务 ID、需求描述补充至提交信息。

3 级：

- 1) 应支持多团队协作场景下的智能分支策略推荐；
- 2) 应具备代码合并冲突预测的能力，并提前给出规避方案；
- 3) 应具备跨版本业务影响追溯能力，支持识别某一版本变更的影响范围；
- 4) 宜支持基于语义的智能冲突解决与版本发布流程自动化。

7.2.7. 智能联调

1 级：

- 1) 应支持 java、js、python、go、c/c++等主流语言的基础日志解析，提取错误信息、异常堆栈关键字；
- 2) 应能识别常见异常类型（如空指针、数组越界、SQL 语法错误），给出基础排查方向；
- 3) 应支持调试断点推荐，基于异常位置提示可能需要断点的代码行；
- 4) 应具备链路片段简单调用并可展示。

2 级：

- 1) 应支持多语言、多框架的异常日志深度分析，提取异常上下文；
- 2) 应支持辅助定位异常根因，给出问题解决方案；
- 3) 应支持分布式链路追踪关联，展示跨服务调用中的异常传播路径；
- 4) 应支持根据报错日志生成调试步骤建议；
- 5) 应支持关联代码历史变更，根据提交历史分析并给出可能引入问题的位置。

3 级：

- 1) 应支持智能化复杂业务场景的根因分析，形成完整根因链路；
- 2) 应能基于 AI 算法预测潜在调试风险；
- 3) 应支持调试过程自动化，模拟参数修改验证；
- 4) 应具备自动生成针对性修复建议，包含代码修改方案、配置调整步骤，支持一键应用等；
- 5) 宜支持关联运维 FAQ 等团队知识库，形成个性化调试策略。

7.2.8. 异常检查

1 级：

- 1) 应支持 java、js、python、go、c/c++等主流编程语言的实时语法错误检测；
- 2) 应能适配编码规范，提示不符合规范的代码（如变量命名不规范、函数过长）；
- 3) 应能识别编译错误（如依赖缺失、方法参数不匹配），实时给出提示；
- 4) 应具备基础安全检测能力，自动规避常见漏洞模式（如 SQL 注入、空指针解引用等）；
- 5) 应支持运行时异常预判（如数组越界、类型转换异常等），给出提示；
- 6) 应支持异常可视化展示。如通过高亮、波浪线、侧边标记等方式，可支持一键跳转功能。

2 级：

- 1) 应支持多语言的深度异常检查，包括逻辑错误、资源泄漏等；
- 2) 应具备框架使用异常的预判能力；
- 3) 应具备异常检查使用能力，支持自定义异常检查规则配置；
- 4) 应支持可关联项目依赖库，自动检查识别 API 使用异常；
- 5) 应支持生成异常风险等级，按优先级提示，支持批量修复简单异常；
- 6) 应支持跨文件异常判断和关联检查。

3 级：

- 1) 应支持多语言、复杂业务场景的异常检查，识别业务逻辑异常；
- 2) 应支持异常影响范围分析，智能评估异常可能导致的业务损失；
- 3) 应支持提供异常预防方案，并结合代码工程自动应用检测和预防；
- 4) 应支持基于线上运行日志、系统监控指标、测试失败数据构建关联分析模型，自动关联异常代码位置与真实线上故障，提供根因定位辅助报告；
- 5) 宜利用 AI 算法（如深度学习、强化学习）构建异常预测模型；
- 6) 宜与代码提交环节、CI/CD 流水线集成，自动触发异常扫描，生成包含风险等级、影响范围、修复优先级的结构化风险评估报告。

7.2.9. 开发自测试

1 级：

- 1) 应支持为简单函数生成单元测试用例；
- 2) 应支持设置断点、查看变量值等基础调试功能；
- 3) 应支持简单测试数据自动生成。

2 级：

- 1) 应支持多语言的场景化测试用例自动生成，覆盖正常、异常、边界场景。确保分支与路径覆盖；
- 2) 应支持分布式链路调试，并能捕获和解析异常堆栈信息；
- 3) 应支持智能自动调用可集成测试框架，支持测试套件生成与批量执行；
- 4) 应支持测试结果与代码关联，自动标记未覆盖代码与潜在缺陷。

3 级：

- 1) 应支持多语言、全链路测试用例生成；
- 2) 应能基于代码变更和缺陷模式，智能生成与优化测试用例集；
- 3) 应支持测试用例智能优化，包括合并重复用例、优先执行高风险用例；

- 4) 应具备自动执行测试并生成测试报告的能力，支持生成报告分析和生成改进方案；
- 5) 应支持自动分析测试失败原因，智能关联代码异常位置，给出修复建议；
- 6) 应能结合 CI/CD 流水线实现自测的自动化。

7.3. 开发者辅助（新增）

7.3.1. 设计辅助

1 级：

- 1) 应具备提供基础设计模板的能力；
- 2) 应能识别基础设计问题，包括不限于接口命名不规范、表字段类型不合理等；
- 3) 应支持自动生成基础设计文档框架；
- 4) 应支持常用字段、接口参数类型建议等设计要素快速录入。

2 级：

- 1) 应支持多技术栈的架构合理性智能校验，并给出校验结果；
- 2) 应能基于业务需求推荐合适的技术方案；
- 3) 应支持在模块拆分优化、接口参数简化、数据库索引设计等方面提供设计优化建议；
- 4) 应支持自定义设计规范的导入并应用；
- 5) 应支持智能生成设计可视化图表包括架构图、模块依赖图、接口调用图等。

3 级：

- 1) 应支持复杂业务场景的智能设计方案生成；
- 2) 应具备基于业务目标动态优化设计方案的能力，支持多维度平衡需求；
- 3) 应具备设计方案的风险预判的能力，提供规避方案；
- 4) 应具备持续演进的能力，结合实际场景生成个性化设计方案；
- 5) 宜支持关联企业业务架构，智能生成研发设计与业务战略一致的方案。

7.3.2. 智能问答

1 级：

- 1) 应支持基础语法问答，可基于内置知识库提供标准化答案；
- 2) 应能解答常用开发工具使用问题（如 Git 提交命令、IDEA 调试技巧）；
- 3) 应支持简单错误排查问答；
- 4) 应支持自然语言提问。

2 级:

- 1) 应支持框架与中间件使用问答;
- 2) 应能解答复杂技术问题, 可提供多维度解答(如代码示例、步骤说明、参考文档链接);
- 3) 应支持项目上下文关联问答;
- 4) 应支持问答历史记录查询, 可收藏常用答案;
- 5) 应能识别模糊提问, 通过追问明确用户需求。

3 级:

- 1) 应支持业务与架构层面的问答;
- 2) 应能结合企业内部知识库、项目文档, 提供个性化解答;
- 3) 应支持复杂问题的分步解答, 引导开发者逐步解决问题;
- 4) 应支持学习团队常见问题与解决方案, 持续优化问答准确性;
- 5) 应支持跨领域问答, 通过跨域知识块优化问答准确性;
- 6) 应能基于问答结果推荐相关学习资源、最佳实践, 辅助开发者能力提升。

7.3.3. 问题分析与修复

1 级:

- 1) 应支持 java、js、python、go、c/c++等主流语言的语法错误分析与修复;
- 2) 应支持基础安全漏洞智能识别和修复方案提供, 支持手动复制应用;
- 3) 应支持自动关联异常提示信息, 快速定位问题位置。

2 级:

- 1) 应支持多语言的性能问题分析, 并提供优化方案;
- 2) 应能智能分析复杂逻辑错误, 定位根因并修复;
- 3) 应支持安全漏洞问题的智能识别和修复方案提供;
- 4) 应支持关联项目依赖库, 自动修复 API 使用错误问题;
- 5) 应支持批量问题修复, 对同类问题一键批量处理;
- 6) 应能自动生成问题修复报告, 包含修复前后对比、影响范围评估等。

3 级:

- 1) 应支持业务逻辑缺陷智能分析与修复方案自动输出;
- 2) 应能学习历史问题修复经验, 生成修复方案;
- 3) 应支持跨模块问题修复;
- 4) 应具备可预测修复方案可能引入的新问题, 提供风险规避建议。

7.3.4. 冲突解决

1 级：

- 1) 应支持对三路合并的冲突分析和自动化冲突解决；
- 2) 应支持对代码补丁在工作应用时冲突的自动化解决；
- 3) 应提供可视化冲突展示，如代码自动进行对比展示，高亮展示冲突文件与冲突位置等。

2 级：

- 1) 应支持对两个分支合并时的冲突的自动化解决；
- 2) 应支持分支合并冲突解决的验证；
- 3) 应具备自动生成冲突回滚方案，若合并后出现问题支持快速回滚。

3 级：

- 1) 应支持分支基线变更，并能自动解决提交应用过程中的冲突，并支持更新提交；
- 2) 应支持分支基线变更的冲突解决的验证；
- 3) 应具备智能预测潜在冲突能力，支持在开发过程给出规避建议。

7.3.5. 质量（实时）助手

1 级：

- 1) 应支持在代码编写、保存时实时更新质量指标，无需手动触发；
- 2) 应能展示当前文件的质量评分，直观反映代码质量水平。

2 级：

- 1) 应支持多维度质量指标实时监测，自定义质量阈值与标准；
- 2) 应支持质量问题精准定位，给出具体优化步骤与代码示例；
- 3) 应能展示质量趋势变化（如当前文件质量较上一版本的提升 / 下降情况）；
- 4) 应支持关联项目整体质量水平，给出个性化改进建议；
- 5) 应支持质量指标可视化展示。

3 级：

- 1) 应支持业务质量实时监测，关联业务目标评估质量；
- 2) 应能智能预测质量风险趋势并自动发出提醒；
- 3) 应支持智能生成个性化质量提升方案（策略）；
- 4) 应支持关联研发度量体系，智能生成团队与个人效能评估。

应支持解析代码的性能瓶颈与安全隐患，提供根因分析；

7.4. 智能评审

7.4.1. 代码质量检查

7.4.2. 代码安全检查

1 级：

- 1) 应支持常见代码安全漏洞的自动化扫描（如 SQL 注入、XSS、硬编码凭证）；
- 2) 应支持指定代码文件/目录的手动触发扫描；
- 3) 应支持扫描结果的基础详情展示（漏洞类型、代码行、严重级别）；
- 4) 应实现扫描过程的实时状态反馈。

2 级：

- 1) 应支持研发流程嵌入式扫描（代码提交、分支合并等环节自动触发）；
- 2) 应支持多语言代码的安全扫描（覆盖 Java、Python、Go 等主流开发语言）；
- 3) 宜支持漏洞的批量标记与分配（将漏洞关联至对应开发人员）；
- 4) 宜支持扫描结果的差异对比（对比不同版本代码的漏洞变化）。

3 级：

- 1) 应支持基于代码上下文的智能漏洞识别；
- 2) 应支持跨代码模块的关联漏洞扫描（识别依赖调用链中的安全风险）；
- 3) 应支持实时增量扫描；
- 4) 宜支持与开发 IDE 的集成，风险实时提醒。

7.4.3. 检查规则

1 级：

- 1) 应提供预设的通用安全规则库（覆盖 OWASP Top 10 基础漏洞类型）；
- 2) 应支持规则的启用/禁用配置；
- 3) 应展示规则对应的漏洞说明与风险等级。

2 级：

- 1) 应支持自定义安全规则的编写与导入；
- 2) 宜支持规则与漏洞严重级别的关联映射触发。

3 级：

- 1) 应支持基于 AI 的规则自学习及更新；
- 2) 宜支持规则根据业务场景的智能推荐。

- 1) 应支持架构与设计合理性分析，识别循环依赖、包设计问题、反模式及性能问题等。

7.4.4. 文档/交付物评审

7.4.5. 评审建议

7.4.6. 代码溯源

7.5. 辅助 CI/CD

1 级：

- 1) 应支持根据代码变更，自动触发基础的编译、打包、部署任务；
- 2) 应能对数据库脚本、配置项的变更进行基础的管理与版本记录，并支持随应用版本同步执行或回滚。

2 级：

- 1) 应支持多环境（开发、测试、生产）的自动化部署与配置管理，并能根据环境差异自动化执行对应的数据库变更脚本与配置项更新；
- 2) 应能将数据库设计与配置管理集成到 CI/CD 流程中，支持作为质量规范对变更的自动化检查（如语法检查、合规性校验）。

3 级：

- 1) 应能基于数据驱动和策略规则，智能编排和优化包含应用、数据库、配置在内的完整发布流程（如变更顺序、灰度策略）；
- 2) 宜能预测因数据库变更或配置更新引发的部署风险（如性能下降、服务依赖中断），并提供优化建议或自动回滚方案。

8. 能力应用

8.1. 企业研发管理

8.1.1. 组织管理

1 级：

- 1) 应明确研发智能化相关工作的责任部门或牵头角色，负责智能化工具和能力在研发活动中的引入、支持与基础管理；
- 2) 应明确研发智能化应用过程中人工决策与智能辅助的基本责任边界，确保研发结果具备明确责任主体；
- 3) 宜支持使用智能化工具开展研发辅助活动；
- 4) 宜对研发智能化应用进行基础协调与支持，避免对现有研发活动产生冲突。

2 级：

- 1) 应明确研发智能化相关组织角色及职责分工，支撑智能能力在需求分析、设计、开发、测试等研发流程中的系统化应用；
- 2) 应建立研发、测试、运维、安全等相关角色在研发智能化应用中的协同机制，支撑跨角色、跨阶段的人机协同；
- 3) 应明确人类角色与智能能力在研发活动中的分工，区分建议、执行与确认等不同参与模式；
- 4) 宜根据人机协同研发需求，对现有组织职责、协作方式或管理流程进行调整；
- 5) 宜支持对研发智能化相关组织运行情况进行评估和优化。

3 级：

- 1) 应建立面向研发智能化的组织运行机制，统一规划、推进和协调智能能力在研发体系中的应用；
- 2) 应形成稳定的人机协同研发组织模式，明确人类与智能能力在研发决策、执行、评审等环节中的职责边界；
- 3) 应支持研发智能化相关组织根据智能能力演进和应用规模变化进行动态调整；
- 4) 宜建立持续优化研发智能化组织形态和协作机制的运行机制。

8.1.2. 制度管理

1 级：

- 1) 应在研发管理相关制度中，对智能化工具和能力的使用作出原则性规定；

- 2) 应明确研发人员在使用智能化工具过程中的基本行为规范与要求；
- 3) 宜对研发智能化应用涉及的数据使用、成果引用等关键事项制定基本要求。

2 级：

- 1) 应建立覆盖研发活动全过程的研发智能化相关管理制度或规范，明确智能能力在各研发环节中的使用要求；
- 2) 应明确智能能力应用过程中数据安全、知识产权、成果责任归属等管理要求；
- 3) 应支持对研发智能化应用过程进行检查、审计或追溯；
- 4) 宜将研发智能化相关制度纳入企业现有研发管理、质量管理或安全管理体系，实现制度协同。

3 级：

- 1) 应形成系统化的人机协同研发制度体系，制度设计能够适配智能能力的持续演进；
- 2) 应通过制度明确智能能力在研发活动中的授权机制、决策边界与责任承担方式；
- 3) 应支持基于研发数据和度量结果，对研发智能化相关制度进行评估和优化；
- 4) 宜形成制度与研发运行数据联动的动态调整机制。

8.2. 智能能力应用

8.2.1. 智能化工具集成

1 级：

- 1) 应支持在主流 IDE（如 VS Code、IntelliJ IDEA）或 Web 端一键安装/启用工具，无需人工配置复杂依赖；
- 2) 应具备工具版本检测与更新提示，确保与 IDE、运行时版本兼容；
- 3) 应提供工具使用指引（含快捷键、菜单入口、首次使用向导），帮助开发者快速上手。

2 级：

- 1) 应支持在制品库或插件市场中统一管理工具版本、权限与灰度发布策略；
- 2) 应能基于项目技术栈自动推荐缺失的智能化工具组合（如 Spring 项目推荐 MyBatis 智能补全、SQL 优化插件）；
- 3) 应支持工具间数据互通与上下文共享（如代码生成结果可直接被测试用例生成工具复用）
- 4) 应具备工具运行健康度监控，实时上报性能指标（CPU、内存占用、响应时延）并给出阈值告警；

- 5) 支持工具热插拔，升级或回滚无需重启 IDE 或中断当前任务。

3 级：

- 6) 应支持基于组织级画像（业务域、技术栈、合规要求）智能编排工具链，实现“开箱即用”的端到端智能化 workflow；
- 7) 应具备工具能力评估与持续优化机制，依据真实使用数据（采纳率、耗时、缺陷关联度）自动调整推荐权重；
- 8) 应支持跨 IDE、跨平台（本地、云端、容器化环境）工具配置同步与一致性校验；
- 9) 具备工具能力开放框架，允许企业自定义扩展并以标准化接口接入生态。

1 级：

- 10) 应支持在主流 IDE（如 VS Code、IntelliJ IDEA）或 Web 端一键安装/启用工具，无需人工配置复杂依赖；
- 11) 应具备工具版本检测与更新提示，确保与 IDE、运行时版本兼容；
- 12) 应支持研发人员按需调用不同智能化能力。

2 级：

- 1) 应建立面向研发活动的智能化工具体系，支持多类智能能力在研发流程中的统一接入与使用；
- 2) 应支持导入企业架构规范/代码规范/安全基线等组织级知识库，驱动工具生成符合标准的交付物；
- 3) 应支持跨工具的智能上下文共享，如代码生成工具可自动获取智能评审的缺陷历史、测试生成工具可读取代码解析的依赖图谱等；
- 4) 宜支持基于研发角色或研发阶段，对智能化工具的可用能力进行差异化配置。

3 级:

- 1) 应构建平台化的智能化工具体系，对研发智能能力进行统一管理、配置、调度与治理；
- 2) 应具备基于生产反馈数据自动优化工具参数与调用策略的能力；
- 3) 应支持基于研发目标的自主工具链编排，AI 可动态选择最优工具组合并调度执行顺序；
- 4) 应支持智能化工具体系与研发流程、研发数据和研发度量体系的协同运行。

8.2.2. 人机协同流程治理（新增 2）

1 级:

- 1) 应完成对现有研发流程的 AI 能力嵌入点梳理；
- 2) 应在代码合并、发布申请等关键节点增加 AI 检查环节；
- 3) 应明确研发人员在使用智能化工具过程中的基本操作边界与责任。

2 级:

- 1) 应支持根据项目类型、团队成熟度动态调整 AI 介入策略，如创新项目降低 AI 生成代码比例，维护项目提高 AI 自动化率；
- 2) 应支持对人机协同流程的执行过程进行管理和监控，确保流程可控。

3 级:

- 1) 应通过流程规则明确智能能力在研发决策、执行和评审等环节中的授权范围、约束条件和兜底机制；
- 2) 应支持根据智能能力成熟度和运行效果，对人机协同流程中的职责分工进行动态调整。

8.3. 研发度量智能化（新增 2）

8.3.1. 智能数据采集

1 级:

- 1) 应能够从版本控制系统（如 Git/SVN）、项目管理工具（如 Jira/禅道）、CI/CD 流水线（如 Jenkins/devops）等核心研发工具中，实现数据的自动化采集与同步，为度量建立统一的数据基础；
- 2) 应支持采集研发过程中 AI 辅助工具的使用数据（如代码生成、评审、测试、文档编写等场景），通过多维度指标评估 AI 工具的应用效果与价值；
- 3) 应利用预设规则，对采集到的数据进行完整性检查（如关键字段是否为空）和格式校验（如时间格式是否标准），并完成基础清洗（如去除重复记录），确保原始数据的准确与可靠；

- 4) 宜支持对提交日志、代码评论等非结构化文本，进行基础的关键词提取与分类（如识别“缺陷修复”、“性能优化”等类型），将这些文字描述转化为可统计、可分析的量化维度。

2 级：

- 5) 应支持通过自然语言处理（NLP）技术，对任务、缺陷等文本进行智能解析与分类，自动生成结构化标签；
- 6) 应能基于业务实体（如需求/缺陷 ID），自动关联并构建跨系统、跨阶段的全链路数据轨迹；
- 7) 宜具备数据质量智能监控能力，可自动检测异常值、规则违背及数据特征分布的重大变化（如缺陷模块分布突变）并预警；
- 8) 可根据预设的分析场景或性能目标，动态调整数据采集的广度、深度与频率。

3 级：

- 1) 应支持基于多模态 AI 模型，对会议录音、设计草图等富媒体资料进行智能分析与关键信息提取；
- 2) 应能够预测未来关键事件的数据需求，并主动发起或增强相关数据的采集过程；
- 3) 宜实现“AI 就绪型数据”的自动治理，确保数据集的准确性、偏差可控，并适用于特定 AI 分析场景；
- 4) 可通过智能体技术，模拟用户与系统交互，主动探索并采集难以通过接口获取的深层流程数据。

8.3.2. 智能化应用价值度量体系

1 级：

- 1) 应支持通过预置规则或模型，对研发过程中 AI 工具（如代码补全、生成、漏洞采集）的使用数据进行度量，生成基础的应用频次、采纳率等效率指标。
- 2) 应能基于简单规则（如固定阈值比如达标/未达标），对关键指标进行健康度判断与基础告警的度量。
- 3) 宜支持对 AI 工具（如代码生成、评审辅助）的应用效果进行度量，评估其在替代重复劳动、提升单点任务效率方面的价值。

2 级：

- 1) 应能应用统计分析或机器学习模型，自动识别并呈现不同度量指标间的关联关系（如代码生成量与缺陷率、评审效率的联动），支撑对研发效能的系统性洞察。
- 2) 应建立度量追踪机制，通过对比分析，量化评估智能化能力嵌入标准流程后，对整体研发效

率与质量的关键指标提升效果。

- 3) 宜采用算法模型，根据团队历史数据动态优化指标的基准阈值，并能基于项目特征智能推荐初始度量方案。

3 级：

- 1) 应构建预测性度量模型，能够基于研发过程数据（如代码活动、需求流转）前瞻性地预测交付风险、质量趋势或业务影响。
- 2) 宜应用因果推断等高级分析技术，尝试量化关键研发实践或智能化改进措施对最终业务成果（如用户满意度、市场响应速度）的贡献度。
- 3) 应能模拟（What-If）不同技术或管理决策对度量体系的综合影响，为研发战略调整提供数据驱动的智能推演与决策支持。

8.3.3. 智能分析

1 级：

- 1) 应支持基于预置规则，对关键度量指标的异常波动进行自动告警。
- 2) 应能自动生成标准化分析报告，汇总展示核心指标的状态、历史趋势及基础对比。
- 3) 宜支持按项目、阶段或研发对象，对研发数据进行分类汇总和对比分析；
- 4)
- 5) 应支持对采集到的研发度量数据进行基础统计分析和可视化展示，反映研发活动的基本状态
- 6) 宜具备对 AI 工具使用情况（如采纳率、满意度）的专项分析能力，以支持其单点效能的评估与优化。

2 级：

- 1) 应应用聚类、分类等机器学习算法，自动识别研发过程中的瓶颈或高风险事件（如特定类型的任务延期）。
- 2) 应支持利用统计分析或机器学习方法，对研发过程数据进行关联分析与模式识别
- 3) 应支持对研发效率、研发质量和研发风险等关键指标进行趋势分析和成因分析
- 4) 应采用时序分析模型，识别指标变化的周期性、趋势性规律，并提供解释。
- 5) 宜能对海量度量数据进行多维度下钻与关联分析，自动定位问题根因或优秀实践的关键特征。
- 6) 宜支持将分析结果与研发对象、流程阶段或智能能力应用情况进行关联

3 级：

- 1) 应建立风险预测模型，能提前预警项目延期、质量滑坡、团队倦怠等潜在问题。

- 2) 应支持对关键研发指标进行智能预测与情景模拟，评估不同决策或改进策略的潜在影响
- 3) 应支持将分析结果用于支撑研发决策和流程优化
- 4) 应用机器学习算法，深度分析研发活动中人员、任务、制品间的复杂网络关系，发现隐性瓶颈或协同问题。
- 5) 应能根据分析结果，自动生成数据驱动的、可操作的改进建议，并评估建议的潜在影响。

8.3.4. 优化与改进
